

COMPSCI701 Assignment 3 Report

Command Pattern	
Role	Type(s)
Invoker	Game (Class): In the <i>play()</i> method, it reads user input, creates the correct Command, and calls appropriate <i>command.execute()</i>. Note: the <i>execute()</i> method is only called once inside the <i>play()</i> method.
Receiver	Game (Class) : performs actions for <i>MoveCommand</i>, <i>NewGameCommand</i>, <i>SaveCommand</i>, <i>LoadGameCommand</i>, and <i>QuitCommand</i> via these methods - <i>handleMoveCommand()</i>, <i>createNewGame()</i>, <i>saveGame()</i>, <i>loadGame()</i>, and <i>requestQuit()</i> respectively. Note: Also TurnManager (Class) - Receiver for Move: performs move actions via the method <i>executeMoveInTurn()</i> once <i>Game.handleMoveCommand()</i> delegates.
Command	Command (Interface): with the method <i>execute()</i> .
Concrete Command(s)	MoveCommand (Class - Concrete Class) NewGameCommand (Class - Concrete Class) SaveGameCommand (Class - Concrete Class) LoadGameCommand (Class - Concrete Class) QuitCommand (Class - Concrete Class)
Client	Game (Class): acts as the factory of commands with the method <i>createCommand()</i> .

Memento Pattern	
Role	Type(s)
Originator	Game (Class): - Encapsulates the game state (the Board and current turn state). - Creates <i>BoardSnapshot (snapshot)</i> in <i>saveGame()</i> and restores <i>BoardSnapshot (lastSnap)</i> in <i>loadGame()</i>.
Memento	BoardSnapshot (Class): Stores the internal state of the Originator without exposing its implementation - capturing the amount of seeds in P1 and P2's houses and stores, and the currentPlayer at the turn the game is saved.
Caretaker	GameCareTaker (Class): Responsible for storing the Memento (the latest snapshot) or giving it back on load but never modifying them. Also, cleared on a new game.

Departures from Maintainability Advice	
Advice Identification	Justification (50 words max)

Nhu (Nikko) Pham - A3_Report

This is just from PMD Report: Game.java - "A value of 7 may denote a high amount of coupling within the class (threshold: 5)".	Game orchestrates UI, rules and design patterns, so references to <i>Board</i> , <i>TurnManager</i> , <i>GameResult</i> , <i>BoardSnapShot</i> , <i>GameCareTaker</i> , <i>Command</i> and <i>IO</i> are intentional. I considered decoupling (move <i>GameResult</i> into <i>TurnManager</i> ; push save/load into <i>GameCareTaker</i>), but these either overload <i>TurnManager</i> 's responsibilities or blur Memento roles. Therefore coupling = 7 is a reasonable trade-off.